

FlashGuard

A Consistency-First Architecture for High-Scale,
Limited-Inventory E-Commerce

SYSTEM DESIGN DOCUMENT

Design-First, AI-Assisted System Design Hackathon • Flash-sale platform, 10,000 buyers for 100 units

“Absorb the traffic at the edge, control contention at inventory, and make every critical transaction idempotent.”

INVENTORY

Sales $\leq$ Stock

Never oversell

IDEMPOTENCY

Retry $\neq$ Repeat

No duplicate transactions

RECOVERY

Failure $\neq$ Loss

No lost payments

DOCUMENT	System Design Document (HLD • LLD • Database • API • Reliability • Security)
VERSION	1.0
DATE	October 2026
TEAM	[Team name] • [Member names]

Contents

1	Requirements & Assumptions	3
2	System Context Diagram	4
3	High-Level Architecture (HLD)	4
4	Container Diagram	5
5	Component Diagram — Inventory & Reservation Service	5
6	Deployment Diagram	6
7	Database Design & ER Diagram	6
8	Class Diagram	7
9	Purchase / Reservation Sequence — 10,000 → 100	7
10	Payment Sequence Diagram	9
11	Order Sequence Diagram — Recovery Path	9
12	Order & Reservation State Diagrams	10
13	SOLID Mapping	10
14	Design Pattern Mapping	11
15	API Specification	11
16	Scalability & Reliability Design	12
17	Security & Observability Design	12
18	Architecture Decision Records	13
19	AI-Assisted Prototype / Simulation Evidence (Optional)	13
20	AI Usage Note	13
21	Final Presentation & Jury Defence	14

List of Figures

Figure 1	System context: customers, operators and external providers	4
Figure 2	HLD: edge, business services, data and messaging	4
Figure 3	Containers: services, data stores and event bus	5
Figure 4	Components inside the Inventory & Reservation service	5
Figure 5	Deployment: multi-AZ cluster, data tier and observability	6
Figure 6	Entity-relationship diagram (14 entities)	6
Figure 7	Class diagram: inventory, payment and order modules	7
Figure 8	From 10,000 requests to at most 100 reservations	8
Figure 9	Purchase / reservation sequence	8
Figure 10	Payment sequence with idempotency, timeout and compensation	9
Figure 11	Order creation with Order Service outage (≈30 s)	9
Figure 12	Reservation state machine	10
Figure 13	Order state machine	10

1 Requirements & Assumptions

SALESTORM is a flash-sale platform for limited-stock products. In the reference scenario **10,000 customers** press *Buy Now* within seconds for only **100 units**. The challenge is not serving 10,000 customers; it is deciding correctly which 100 of them own the scarce resource while payments, orders and individual services may be slow, duplicated or failing. FlashGuard answers with one principle, proven through three guarantees.

INVENTORY GUARANTEE Sales ≤ Stock Never oversell: 10,000 requests, 100 units, at most 100 reservations.	IDEMPOTENCY GUARANTEE Retry ≠ Repeat The same request key returns the original result, never a second transaction.	RECOVERY GUARANTEE Failure ≠ Loss A successful payment is never lost; it always ends in a valid order.
---	--	--

1.1 Functional requirements

ID	Requirement	ID	Requirement
FR-01	Customer authentication	FR-11	Duplicate-payment prevention
FR-02	Product discovery	FR-12	Payment timeout, retry & reconciliation
FR-03	Product availability check	FR-13	Order creation
FR-04	Cart management	FR-14	Order state management
FR-05	Buy Now	FR-15	Shipment management
FR-06	Inventory reservation	FR-16	Notifications
FR-07	Reservation expiry	FR-17	Delivery tracking
FR-08	Reservation release	FR-18	Failure recovery
FR-09	Checkout	FR-19	Audit logging
FR-10	Payment processing	FR-20	Monitoring & tracing

1.2 Non-functional requirements

Quality	Strict guarantee (must always hold)	Target / goal (validated by test, not claimed)
Consistency	Stock never negative; successful reservations ≤ available stock (100).	—
Idempotency	One reservation, payment and order per idempotency key.	—
Reliability	A successful payment always results in exactly one order.	Order Service outage of 30 s recovered without manual action.
Scalability	Services are stateless and scale horizontally.	≈10,000 req/s normal; strategy for up to ≈500,000 req/s at peak.
Performance	—	Reservation decision p95 < 300 ms under flash-sale load (target).
Availability	Notifications and shipping never block checkout.	Browse and cart degrade gracefully if dependencies fail.
Security	TLS everywhere; no raw card data stored; authenticated APIs.	Abusive traffic filtered at the edge.
Observability	Every transaction traceable end to end by correlation IDs.	Alerts on invariant breach, consumer lag, breaker state.

1.3 Assumptions & constraints

#	Assumption	#	Assumption
A1	One hot SKU with 100 units; maximum 1 unit per customer.	A5	Kafka delivers at-least-once; consumers are idempotent.
A2	Payment gateway accepts a unique transaction reference and supports status queries.	A6	Redis is a cache only; SQL is the single source of truth for stock.
A3	Reservation TTL is 5 minutes (configurable).	A7	Single region, multi-AZ; disaster recovery is by backup and restore.
A4	Practical test: 95 % payment success, 5 % failure, 2 % duplicates, Order Service down 30 s.	A8	Prototype is optional; the design is justified by reasoning first.

2 System Context Diagram

FlashGuard is a single system boundary. Customers use the web or mobile app. Three external parties are involved: a **payment gateway** (charge and status query), a **shipping partner** (fulfilment and tracking) and a **notification provider** (email, SMS, push). Operations engineers observe the platform through dashboards and alerts.

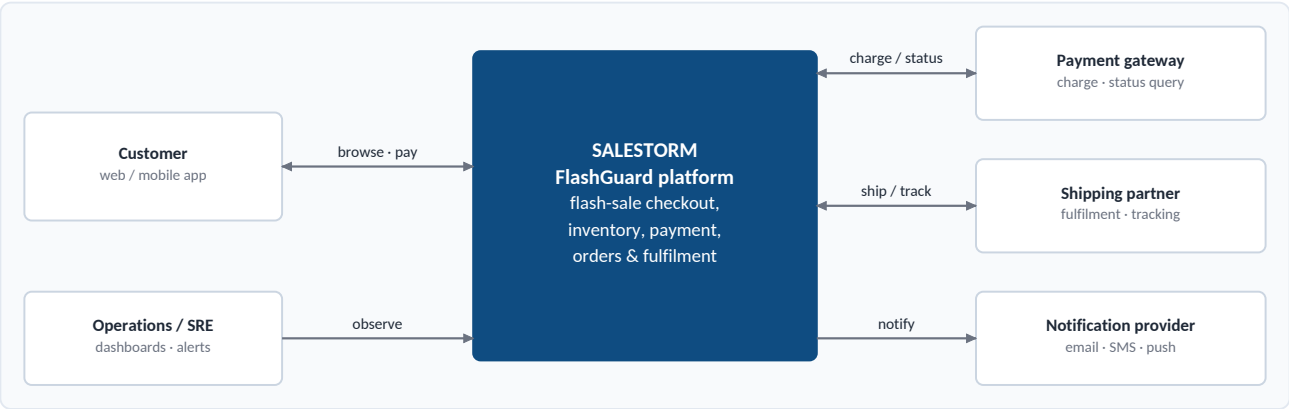


Figure 1 System context: customers, operators and external providers

3 High-Level Architecture (HLD)

Traffic is absorbed in layers. The edge removes static and abusive load, the API gateway authenticates and rate-limits, and only valid requests reach stateless business services. **Inventory & Reservation** is the only component allowed to change stock.

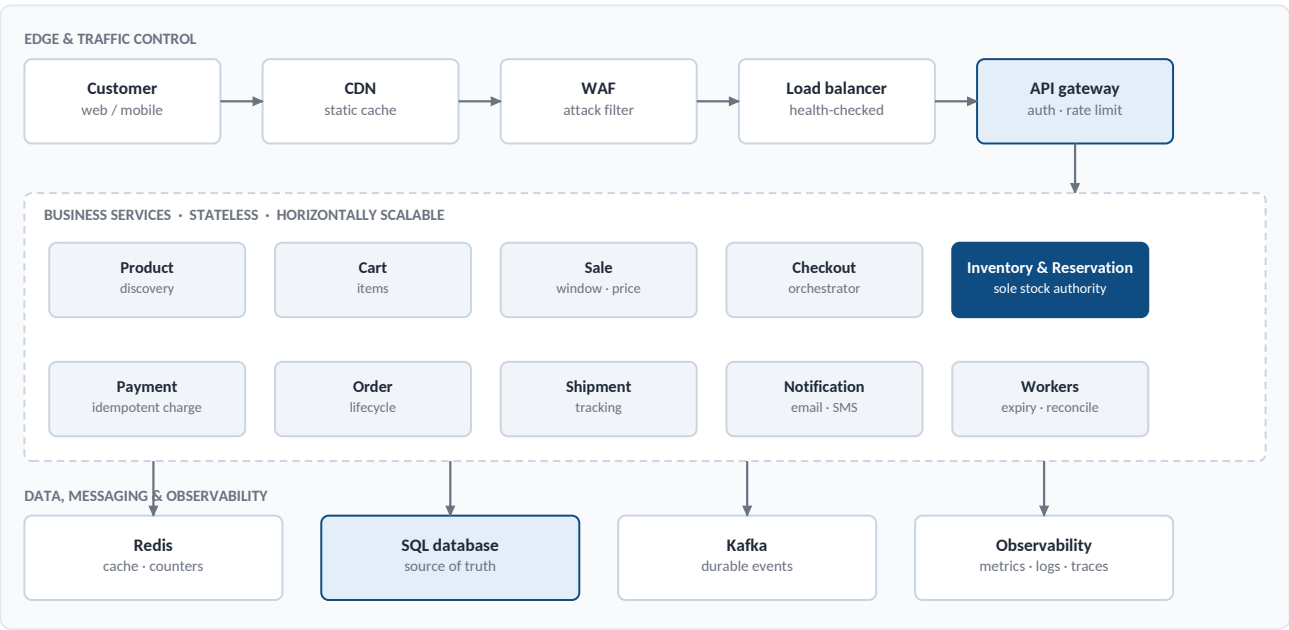


Figure 2 HLD: edge, business services, data and messaging

Component	Responsibility	Communication	If it fails
CDN · WAF · load balancer	Cache static content, filter attacks, spread traffic.	Sync	Origin keeps serving; unhealthy nodes removed.
API gateway	Authentication, rate limiting, validation, routing.	Sync	Replicated; client retries are safe.
Inventory & Reservation	Sole authority over stock changes.	Sync	Fails fast; no partial state (one commit).
Payment	Idempotent charge; status reconciliation.	Sync + async	Circuit breaker; reconcile later.
Order	Order creation and lifecycle.	Async (events)	Events retained in Kafka and redelivered.
Kafka · Redis	Durable events between services; cache and rate counters.	Async · sync	Outbox retries; SQL fallback, Redis never authoritative.

4 Container Diagram

Each business service owns its data store. Services call each other **synchronously only where the customer needs an immediate answer** (reserve, pay) and communicate through Kafka for everything else.

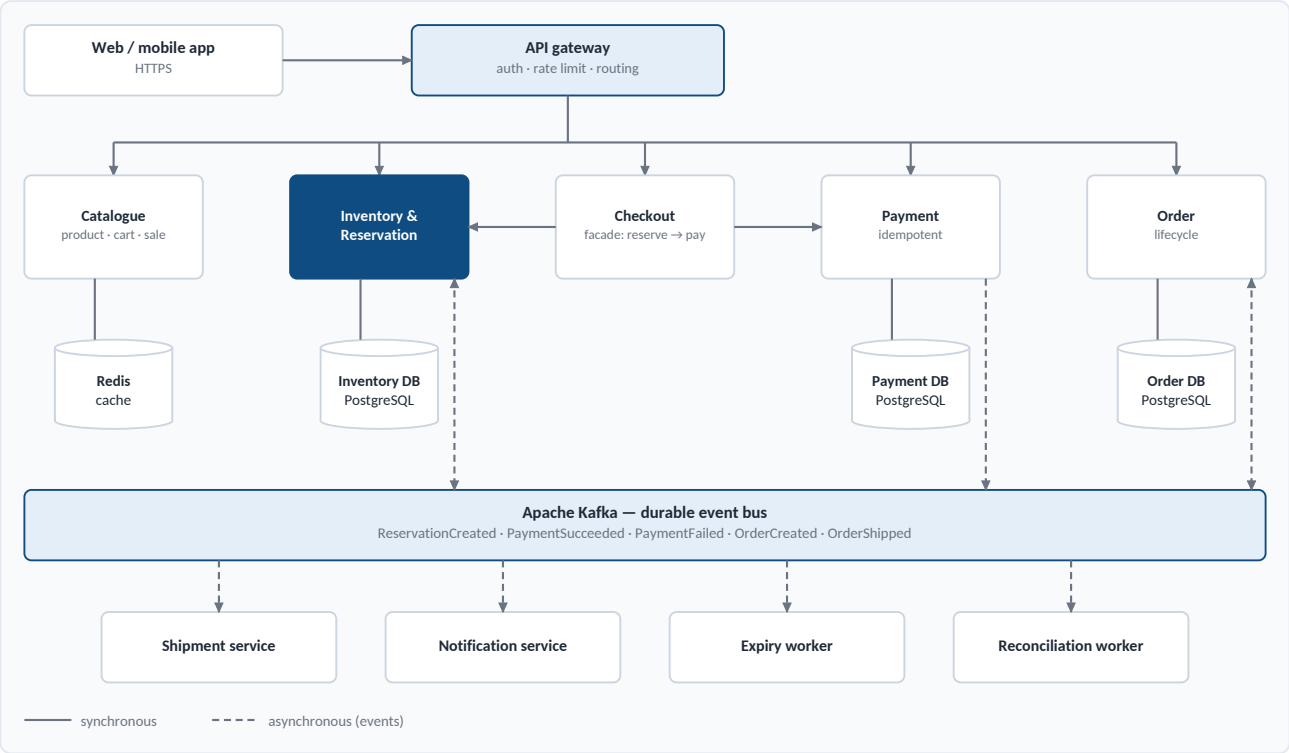


Figure 3 Containers: services, data stores and event bus

5 Component Diagram — Inventory & Reservation Service

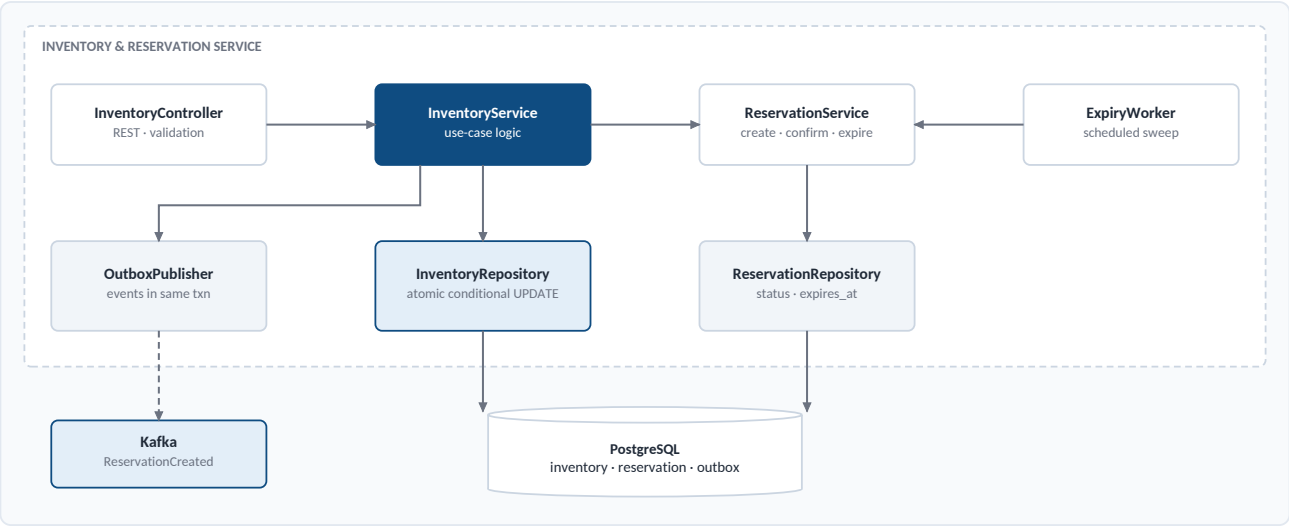


Figure 4 Components inside the Inventory & Reservation service

Component	Responsibility
InventoryController · InventoryService	Validate input and idempotency key; use-case logic; sole caller of the atomic stock update.
ReservationService	Creates, confirms and expires reservations; one reservation per idempotency key.
Repositories · OutboxPublisher · ExpiryWorker	Atomic conditional UPDATE; events written in the same transaction; scheduled expiry sweep.

6 Deployment Diagram

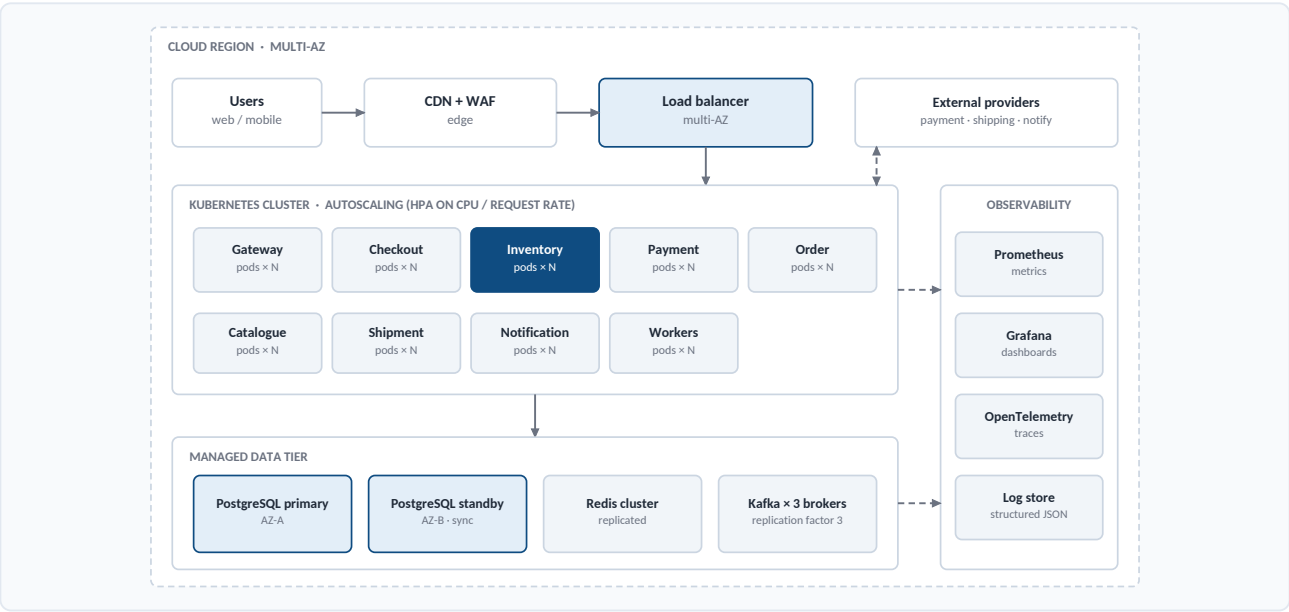


Figure 5 Deployment: multi-AZ cluster, data tier and observability

Stateless pods autoscale on CPU and request rate. PostgreSQL runs a primary with a synchronous standby in a second zone; Kafka has three replicated brokers. Hot-SKU stock stays on the primary.

7 Database Design & ER Diagram

The fourteen mandatory entities are modelled below; **INVENTORY** and **INVENTORY_RESERVATION** (outlined) are consistency-critical. **PAYMENT** keeps *reservation_id* as business key because a payment exists before its order.

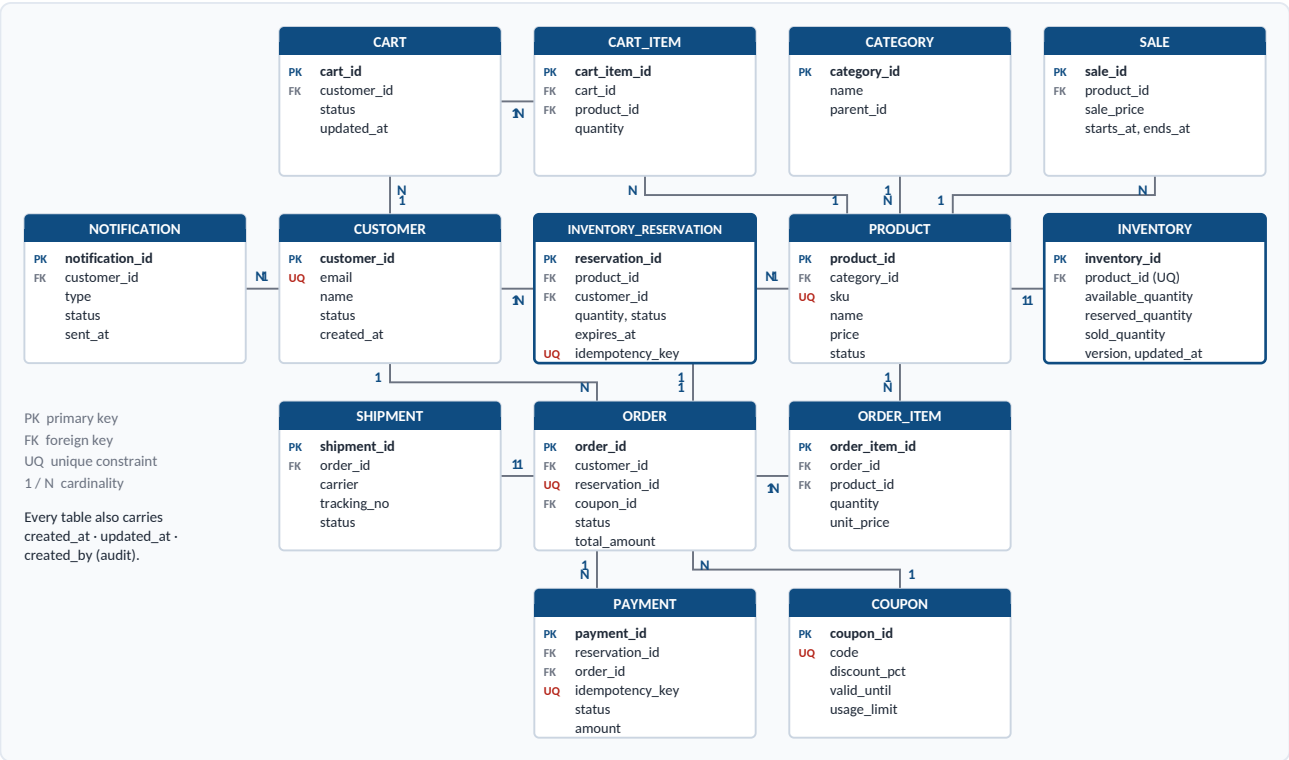


Figure 6 Entity-relationship diagram (14 entities)

Constraint / index	Purpose
CHECK (available_quantity ≥ 0)	Second line of defence: the database itself rejects negative stock.
CHECK (available + reserved + sold = total_stock)	Stock invariant; verified continuously by a monitor.
UNIQUE (idempotency_key) on reservation and payment	Duplicate requests cannot create duplicate rows.
UNIQUE (reservation_id) on ORDER	Order creation is idempotent: one order per reservation.
INDEX (status, expires_at) on reservation	Fast expiry sweep without scanning the table.
version column on INVENTORY	Optimistic check for confirm / release updates and audit.

8 Class Diagram

Each class has one reason to change. Services depend on interfaces (repositories, *PaymentStrategy*), so providers and storage can change without touching business logic.

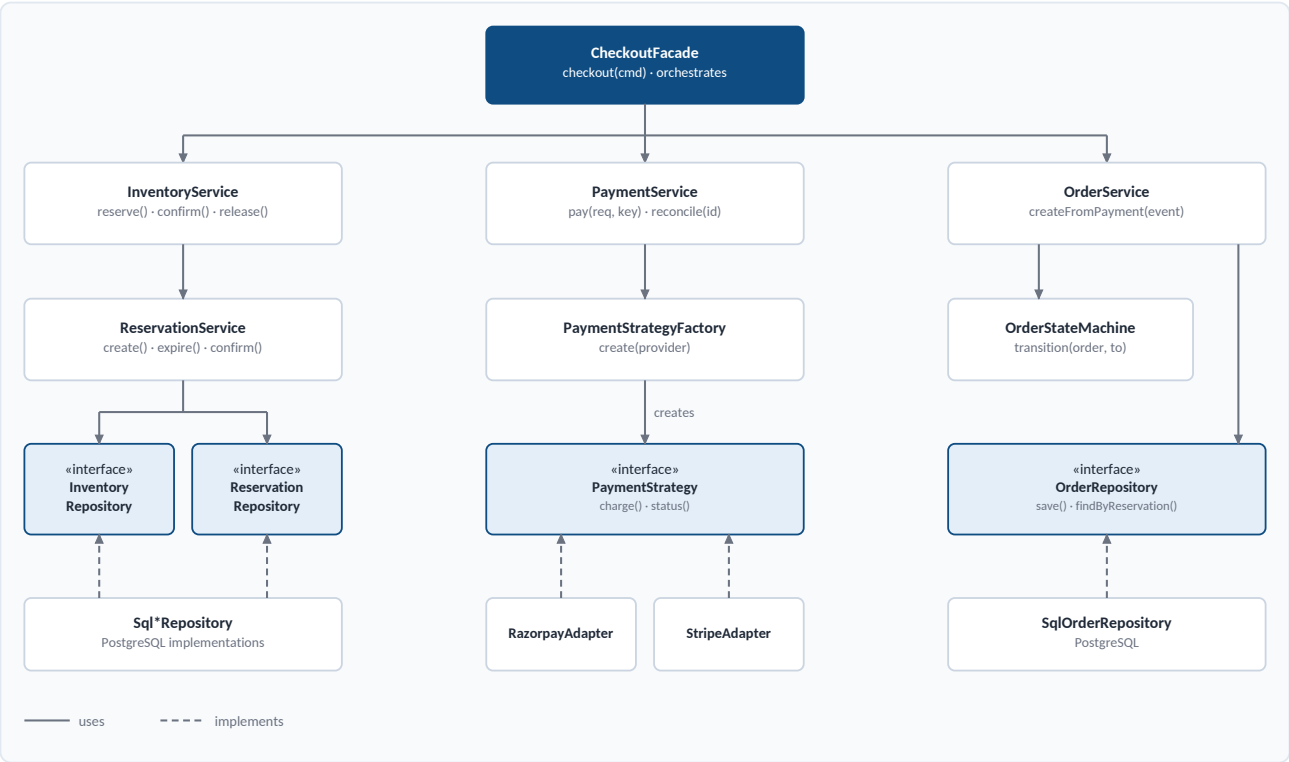


Figure 7 Class diagram: inventory, payment and order modules

9 Purchase / Reservation Sequence — 10,000 → 100

The guarantee does not depend on how many requests arrive. It depends on **one atomic SQL statement**. Two requests for the last unit are serialised by the database: one sees *available_quantity* = 1 and succeeds, the other sees 0 and is told the item is sold out.

```
UPDATE inventory
SET available_quantity = available_quantity - :qty,
    reserved_quantity = reserved_quantity + :qty,
    version = version + 1, updated_at = now()
WHERE product_id = :pid AND available_quantity >= :qty; -- rows affected: 1 = RESERVED, 0 = OUT_OF_STOCK
```

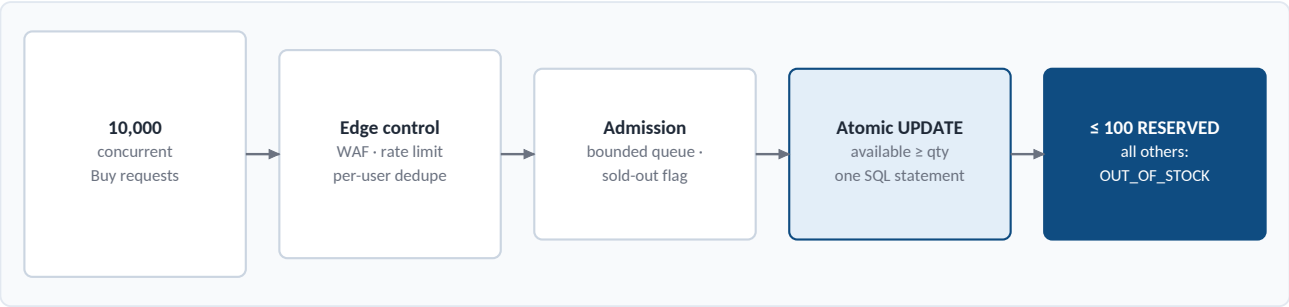


Figure 8 From 10,000 requests to at most 100 reservations

Transaction boundary. One short transaction: idempotency check, conditional UPDATE, INSERT reservation (status RESERVED, expires_at = now + 5 min) and INSERT outbox event, then COMMIT. **No external call happens inside it**; the payment gateway is called only after the commit, so no database lock is ever held while waiting on a slow third party.

Approach	Behaviour under 10,000 contenders	Decision
Pessimistic lock (SELECT ... FOR UPDATE)	Requests queue on one row lock; long waits, timeouts and deadlock risk.	Rejected
Optimistic version check	Most requests fail and retry, creating a retry storm on the hot row.	Used only for confirm / release
Atomic conditional UPDATE	One statement per request, no retries, no lock held across steps.	Selected

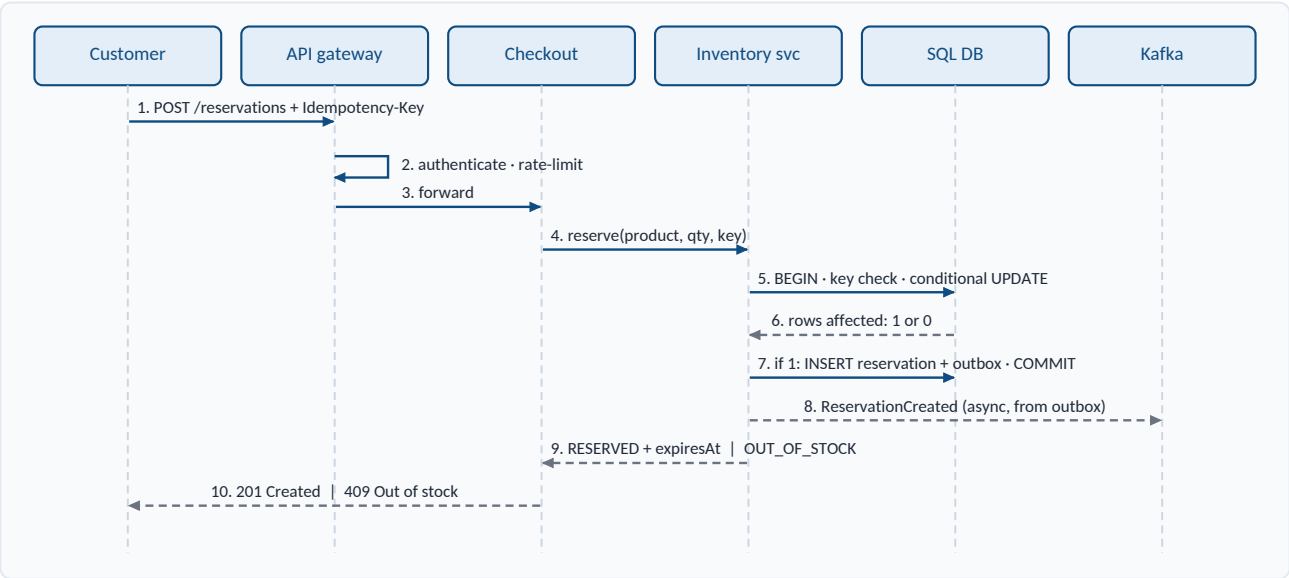


Figure 9 Purchase / reservation sequence

Expiry. A background worker finds RESERVED rows with expires_at in the past and, in one transaction, marks them RELEASED and moves the unit back (available + 1, reserved - 1). Reservations in PAYMENT_PENDING are skipped until payment is resolved or a hard limit is reached.

10 Payment Sequence Diagram

Payment is the least reliable dependency, so the design never assumes an outcome it has not observed. A timeout means *unknown*, not *failed*.

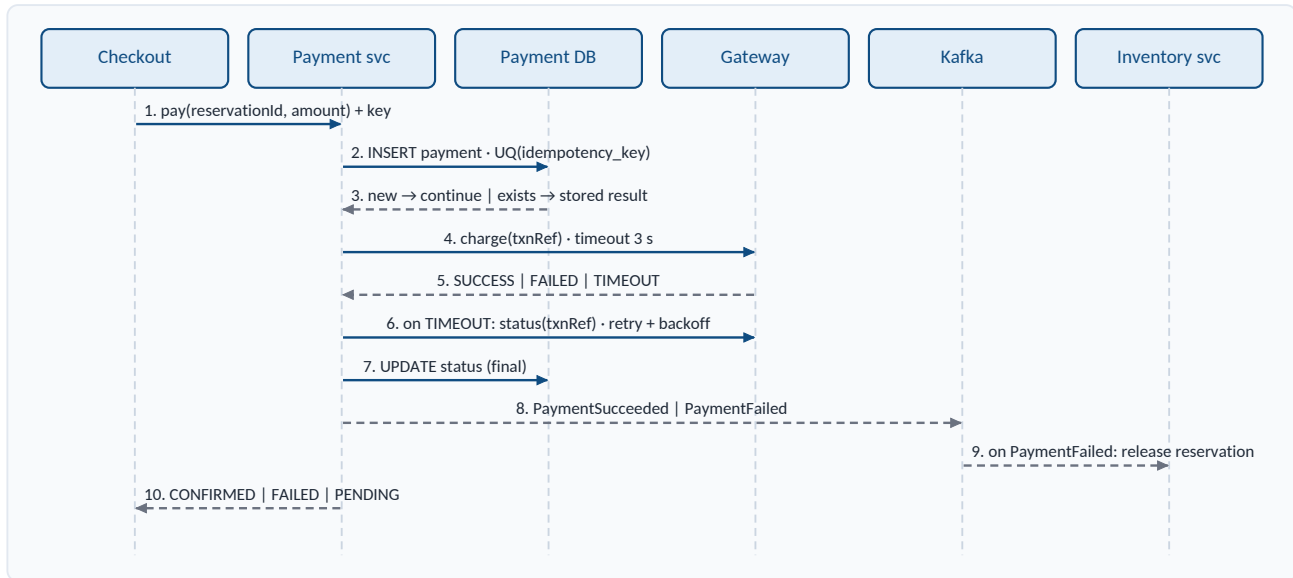


Figure 10 Payment sequence with idempotency, timeout and compensation

Scenario	Handling	Customer outcome
1 Payment succeeds	Persist SUCCESS, emit PaymentSucceeded, confirm reservation, create order.	Order confirmed
2 Payment fails	Emit PaymentFailed; Inventory releases the reservation (compensation).	Stock available again
3 Payment times out	Mark UNKNOWN; query gateway by txnRef; decide only on a final status.	"Confirming payment"
4 Duplicate request	Same Idempotency-Key returns the stored result; no second charge.	Charged once
5 Success, Order down	Event retained and retried; see Section 11.	Order created later
6 Gateway down	Circuit breaker opens, calls fail fast; unpaid reservation expires.	"Try again shortly"

11 Order Sequence Diagram — Recovery Path

The payment result and its outbox event are written in the same transaction, so a success can never exist without an event. If Order Service is down, Kafka retains the event and redelivers it. Order creation is idempotent through *UNIQUE (reservation_id)*. A reconciliation worker is the safety net, and messages that keep failing go to a dead-letter topic with an alert.

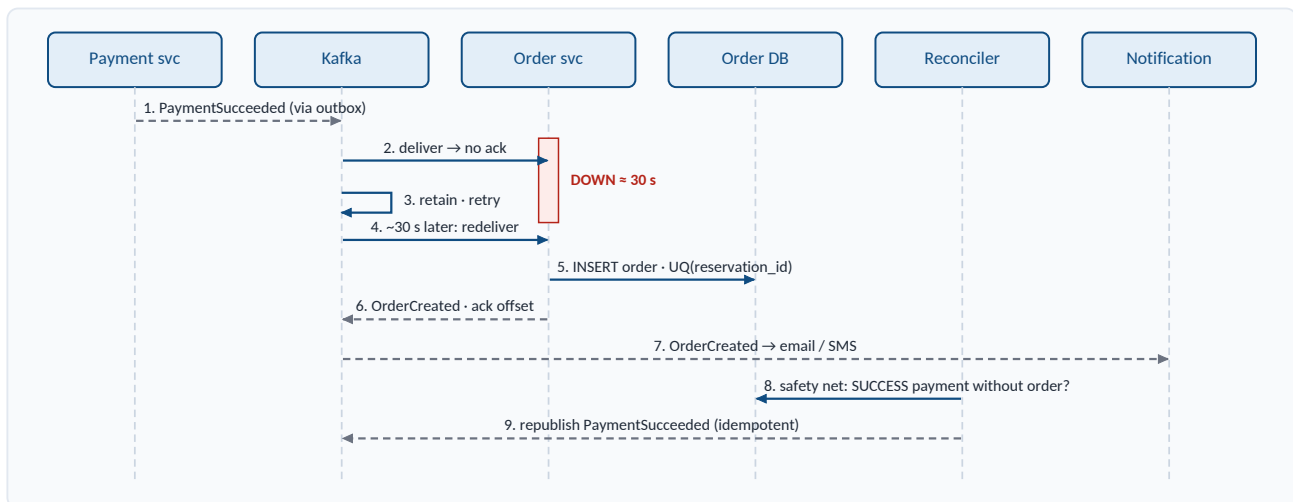


Figure 11 Order creation with Order Service outage (≈30 s)

12 Order & Reservation State Diagrams

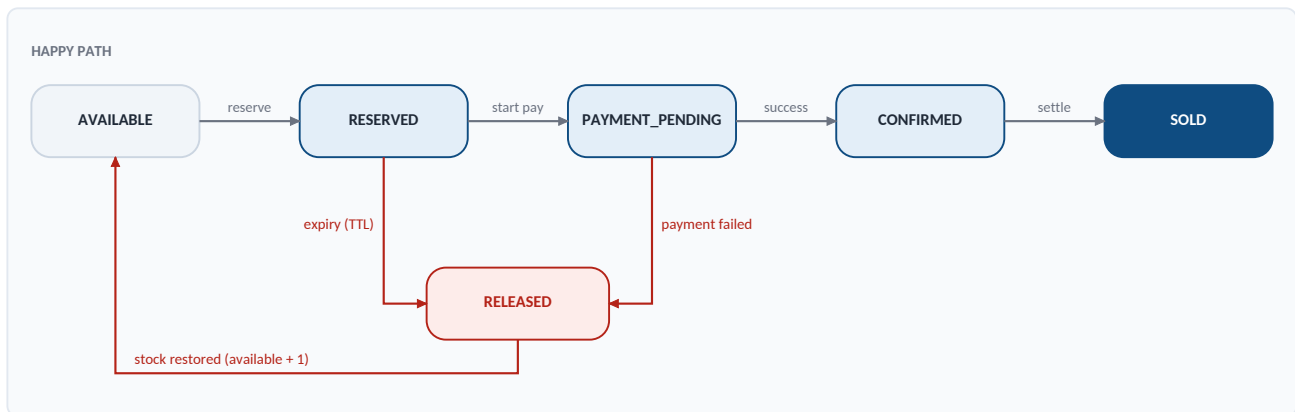


Figure 12 Reservation state machine

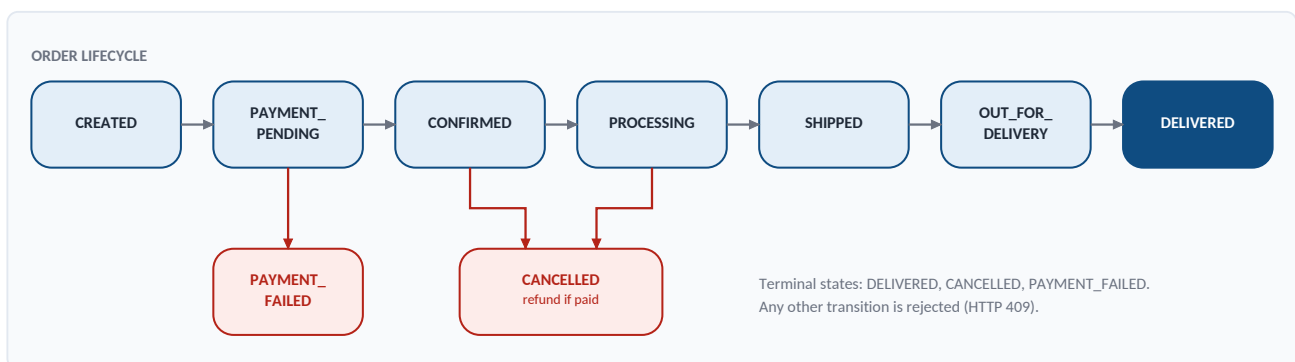


Figure 13 Order state machine

Transition	Allowed owner	Trigger
AVAILABLE → RESERVED	Inventory service	Atomic conditional UPDATE succeeds
RESERVED → PAYMENT_PENDING → CONFIRMED → SOLD	Inventory service	Payment started; PaymentSucceeded event
RESERVED / PAYMENT_PENDING → RELEASED	Inventory service (ExpiryWorker)	TTL elapsed or PaymentFailed event
CREATED → PAYMENT_PENDING → CONFIRMED	Order service	Order created; PaymentSucceeded event
PROCESSING → SHIPPED → OUT_FOR_DELIVERY → DELIVERED	Shipment service (events to Order)	Carrier and warehouse updates
Any other transition	Nobody	Rejected by the State pattern with HTTP 409

13 SOLID Mapping

Principle	Where it appears in FlashGuard	Benefit
Single responsibility	Inventory, Payment, Order and Notification are separate services; ReservationService only handles reservations.	A payment change never risks stock logic.
Open / closed	New payment providers and pricing rules are added as new <i>PaymentStrategy</i> / pricing classes.	No edits to checkout code.
Liskov substitution	RazorpayAdapter and StripeAdapter obey the same contract: charge() returns SUCCESS, FAILED or UNKNOWN.	Any provider can be swapped in.
Interface segregation	Small interfaces: PaymentProcessor, PaymentStatusChecker, RefundProcessor, InventoryRepository, ReservationRepository.	Clients depend only on what they use.
Dependency inversion	PaymentService → PaymentStrategy; ReservationService → repository interfaces; implementations injected.	Testable without a gateway or database.

14 Design Pattern Mapping

Pattern	Problem solved	Where used	Trade-off
Strategy	Several payment providers and pricing rules.	PaymentStrategy, pricing	More small classes.
Factory	Choose the right provider at runtime.	PaymentStrategyFactory	Registry must stay in sync.
State	Invalid lifecycle jumps.	Reservation and order state machines	One class per state.
Observer / events	Notify without coupling to checkout.	Order events → notification, shipment	Eventual consistency.
Adapter	Different external APIs.	Payment and shipping providers	Extra translation layer.
Repository	Keep SQL out of business logic.	Inventory, Reservation, Order	Abstraction over a fixed schema.
Facade	Checkout touches many services.	CheckoutFacade	Risk of a “god” class if unmanaged.
Circuit breaker	A failing gateway exhausts threads.	Payment gateway client	Needs tuned thresholds.

15 API Specification

Method & endpoint	Purpose	Auth	Idempotency	Responses
POST /api/v1/reservations	Reserve stock	Customer JWT	Required key	201, 409 sold out, 429
GET /api/v1/reservations/{id}	Reservation status	Owner	—	200, 404
POST /api/v1/reservations/{id}/confirm	Confirm after payment	Internal	State check	200, 409
POST /api/v1/reservations/{id}/release	Cancel reservation	Owner / internal	State check	200, 409
POST /api/v1/checkout	Orchestrate reserve → pay	Customer JWT	Required key	202, 409
POST /api/v1/payments	Charge reservation	Customer / internal	Required key	200, 202, 402, 503
GET /api/v1/payments/{id}	Payment status	Owner	—	200, 404
POST /api/v1/orders	Create order from payment	Internal	By reservationId	201, 200 (existing)
GET /api/v1/orders/{id}	Order status	Owner	—	200, 404
GET /products · /sales · /cart	Discovery and cart	Customer JWT	—	200

```
POST /api/v1/reservations      Idempotency-Key: 8f92ab12
{ "productId": "P1001", "quantity": 1 }

201 Created { "reservationId": "RES123", "status": "RESERVED", "expiresAt": "2026-10-05T12:20:00Z" }
409 Conflict { "code": "OUT_OF_STOCK", "message": "Sold out", "requestId": "REQ123" }
```

Event catalogue

Event	Producer → Consumers	Topic
ReservationCreated / Released / Expired	Inventory → Checkout, Notification	inventory.reservations
PaymentSucceeded / PaymentFailed	Payment → Inventory, Order, Notification	payments.events
OrderCreated / Confirmed	Order → Shipment, Notification	orders.events
OrderProcessing / Shipped / Delivered	Shipment → Order, Notification	shipments.events

Delivery is at-least-once, keyed by reservationId for per-reservation ordering. Every consumer records processed eventIds and ignores repeats.

16 Scalability & Reliability Design

Layer	Technique for 10,000 → 500,000 req/s
CDN	Product pages, assets and the sale countdown never reach the origin.
WAF + rate limiting	Per-IP and per-user limits and bot rules drop abusive and repeated traffic early.
Gateway and services	Stateless; add instances with autoscaling. No session affinity needed.
Redis	Absorbs hot reads (product, sale state) and holds a short-lived “sold-out” flag.
Admission control	Bounded queue or token bucket in front of inventory; excess gets a fast “sold out / try later” (backpressure).
SQL database	Read replicas for catalogue; stock row stays on the primary; shard by product_id when many SKUs are on sale.
Kafka	Partitions keyed by reservationId; add partitions and consumers for throughput.

Bottleneck and honest trade-off

The limiting factor is the **single hot inventory row**; adding service instances does not help it. The design protects it with edge filtering, a sold-out flag and admission control, so only a bounded number of requests ever reach SQL. Strict consistency means this row is serialised on one primary: we accept that throughput ceiling instead of risking overselling. 500,000 req/s is an architectural target to be validated by load testing, not a measured result.

Failure matrix

Failure	Detection	Response / recovery	Customer sees
Inventory DB down	Health check, connection errors	Fail fast; promote standby; no reservation without commit.	“Try again”
Payment gateway down	Timeouts, error rate	Circuit breaker opens; reservation expires if unpaid.	“Payment unavailable”
Payment timeout	No reply within 3 s	Status query with backoff; reconcile before deciding.	“Confirming payment”
Order Service down	Consumer lag	Kafka retains; retry; reconciliation worker.	“Payment received”
Kafka down	Publish errors	Outbox rows wait and relay on recovery; sync path unaffected.	No impact
Redis down	Cache errors	Fall back to SQL; tighter rate limits.	Slower pages
Duplicate request	Idempotency-Key lookup	Return the original result.	Same response
Duplicate event	eventId already processed	Consumer ignores it.	No impact
Reservation expiry	ExpiryWorker sweep	Release stock in one transaction.	“Reservation expired”
Deadlock / network timeout	DB error code, timeout	Bounded retry with backoff; safe because idempotent.	Usually none
Partial outage	Health checks, alerts	Degrade non-critical features first (notifications).	Core purchase works

17 Security & Observability Design

Security	Control
Transport	HTTPS (TLS 1.2+) at the edge; mutual TLS between services.
Authentication / authorization	OAuth2 / JWT at the gateway; ownership checks on every reservation, payment and order.
Rate limiting and abuse	WAF rules, per-user token bucket, one active reservation per customer.
Input validation	Schema validation; quantity limited to 1; idempotency key format enforced.
Payment data	Tokenised by the gateway; raw card data is never stored (minimal PCI scope).
Secrets and audit	Secrets in a vault, rotated; append-only audit log of every state change.

Observability	Design
Metrics	Request rate, p95 / p99 latency, error rate, reservation success / failure, payment success / failure, order conversion, Kafka lag, expiry rate, circuit-breaker state (Prometheus + Grafana).
Logs and traces	Structured JSON logs; OpenTelemetry traces; one correlation chain across all services.
Alerts	Stock invariant violated; consumer lag above threshold; breaker open; payment-failure spike; successful payment without an order.

Correlation chain

requestId → reservationId → paymentId → orderId → shipmentId. Any failed transaction can be reconstructed end to end.

18 Architecture Decision Records

ADR	Decision	Alternatives rejected	Trade-off accepted
001	Service-oriented, event-driven downstream.	Single monolith; fully asynchronous flow.	More components to operate.
002	PostgreSQL for inventory, payment, order.	NoSQL for stock.	Vertical limit on the hot row.
003	Atomic conditional UPDATE.	Pessimistic locking; optimistic retries.	Hot-row contention; needs admission control.
004	Kafka with transactional outbox.	Chained synchronous calls.	At-least-once → idempotent consumers.
005	Idempotency keys everywhere.	Trusting clients not to retry.	Extra storage and lookup.
006	Redis for cache only.	Redis as stock authority.	Cache may be briefly stale.
007	Sync reservation, async downstream.	All sync; all async.	Customer waits only for the stock decision.

Each ADR follows Context → Decision → Alternatives → Reasoning → Trade-offs → Consequences in the repository.

19 AI-Assisted Prototype / Simulation Evidence (Optional)

A small load script (Locust or JMeter) fires 10,000 concurrent Buy requests at 100 units, followed by SQL assertions. The simulation is **supporting evidence**; correctness is argued by design, not by test output. Measured results are to be recorded by the team after each run.

Test	Setup	Pass criterion
T1 Oversell	10,000 concurrent requests, stock 100	Reservations = 100; available ≥ 0
T2 Duplicates	2 % repeat the same key	One reservation per key
T3 Expiry	Unpaid reservations pass the TTL	Stock returned; invariant holds
T4 Payment failure	5 % of payments fail	Units released and resold
T5 Order outage	Stop Order Service for 30 s after payments	Exactly one order per paid reservation
T6 Gateway outage	Gateway returns errors	Breaker opens; nothing stuck beyond TTL
T7 Invariant	After every run	available + reserved + sold = 100

20 AI Usage Note

Aspect	Declaration
Tool	[Team to state AI tool and version]
Used for	Drafting documentation, diagram drafts, test scenarios, simulation boilerplate, comparing alternatives.
Reviewed by the team	Every section, each diagram against the requirements, every number and claim.
Not delegated	Architecture decisions, concurrency reasoning, consistency, payment reliability, security, trade-offs.
Validation	Design reasoning, SQL invariant checks, simulation and peer review. AI output is never treated as proof of correctness.

21 Final Presentation & Jury Defence

Time	Focus	Show
30 s	Problem: 10,000 users, 100 units	Funnel (Figure 8)
30 s	Requirements and three guarantees	Section 1
60 s	High-level architecture	Figures 2–3
60 s	10,000 users vs 100 units	Atomic UPDATE and sequence
45 s	Payment and order recovery	Figures 10–11
45 s	LLD, SOLID and patterns	Figure 7, Sections 13–14
30 s	Scalability, reliability, AI validation	Section 16, 19

Likely question	Defensible answer
Why can stock never go below zero?	The decrement and the check run in one atomic statement, backed by CHECK (available_quantity ≥ 0).
Two requests for the last unit?	The database serialises them; one gets rows affected = 1, the other 0 and “sold out”.
Why not lock during payment?	Payment latency is unbounded; a lock held across it would stall every buyer. We commit first.
Payment succeeds, Order Service crashes?	The event is durable and retried; UNIQUE (reservation_id) makes order creation idempotent.
Buy clicked twice?	The idempotency key returns the original reservation; no second row.
Kafka delivers an event twice?	Consumers record eventIds and ignore repeats.
Redis goes down?	Only caching is lost; SQL remains the source of truth, so correctness is unaffected.
Biggest bottleneck?	The hot inventory row, protected by edge filtering, admission control and a sold-out flag.

Closing statement

Traffic can scale horizontally. Inventory correctness cannot be compromised.

Appendix Deliverables checklist

#	Deliverable	Where	#	Deliverable	Where
01	Requirements & Assumptions	Section 1	12	Order / Reservation States	Section 12
02	System Context Diagram	Section 2	13	SOLID Mapping	Section 13
03	HLD Architecture	Section 3	14	Design Pattern Mapping	Section 14
04	Container Diagram	Section 4	15	API Specification	Section 15
05	Component Diagram	Section 5	16	Scalability & Reliability	Section 16
06	Deployment Diagram	Section 6	17	Security & Observability	Section 17
07	Database / ER Diagram	Section 7	18	Architecture Decision Records	Section 18
08	Class Diagram	Section 8	19	AI Prototype / Simulation (optional)	Section 19
09	Purchase / Reservation Sequence	Section 9	20	AI Usage Note	Section 20
10	Payment Sequence	Section 10	21	Final Presentation	Section 21
11	Order Sequence	Section 11			

All 21 deliverables are included. Deliverable 19 (prototype) is optional in the brief and is covered as a validation design.